JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Class FileLock

java.lang.Object
    java.nio.channels.FileLock

All Implemented Interfaces:

AutoCloseable

```
public abstract class FileLock
extends Object
implements AutoCloseable
```

A token representing a lock on a region of a file.

A file-lock object is created each time a lock is acquired on a file via one of the `lock` or `tryLock` methods of the `FileChannel` class, or the `lock` or `tryLock` methods of the `AsynchronousFileChannel` class.

A file-lock object is initially valid. It remains valid until the lock is released by invoking the `release` method, by closing the channel that was used to acquire it, or by the termination of the Java virtual machine, whichever comes first. The validity of a lock may be tested by invoking its `isValid` method.

A file lock is either *exclusive* or *shared*. A shared lock prevents other concurrently-running programs from acquiring an overlapping exclusive lock, but does allow them to acquire overlapping shared locks. An exclusive lock prevents other programs from acquiring an overlapping lock of either type. Once it is released, a lock has no further effect on the locks that may be acquired by other programs.

Whether a lock is exclusive or shared may be determined by invoking its `isShared` method. Some platforms do not support shared locks, in which case a request for a shared lock is automatically converted into a request for an exclusive lock.

The locks held on a particular file by a single Java virtual machine do not overlap. The `overlaps` method may be used to test whether a candidate lock range overlaps an existing lock.

A file-lock object records the file channel upon whose file the lock is held, the type and validity of the lock, and the position and size of the locked region. Only the validity of a lock is subject to change over time; all other aspects of a lock's state are immutable.

File locks are held on behalf of the entire Java virtual machine. They are not suitable for controlling access to a file by multiple threads within the same virtual machine.

File-lock objects are safe for use by multiple concurrent threads.

Platform dependencies

This file-locking API is intended to map directly to the native locking facility of the underlying operating system. Thus the locks held on a file should be visible to all programs that have access to the file, regardless of the language in which those programs are written.

Whether or not a lock actually prevents another program from accessing the content of the locked region is system-dependent and therefore unspecified. The native file-locking facilities of some systems are merely *advisory*, meaning that programs must cooperatively observe a known locking protocol in order to guarantee data integrity. On other systems native file locks are *mandatory*, meaning that if one program locks a region of a file then other programs are actually prevented from accessing that region in a way that would violate the lock. On yet other systems, whether native file locks are advisory or mandatory is configurable on a per-file basis. To ensure consistent and correct behavior across platforms, it is strongly recommended that the locks provided by this API be used as if they were advisory locks.

On some systems, acquiring a mandatory lock on a region of a file prevents that region from being *`mapped into memory`*, and vice versa. Programs that combine locking and mapping should be prepared for this combination to fail.

On some systems, closing a channel releases all locks held by the Java virtual machine on the underlying file regardless of whether the locks were acquired via that channel or via another channel open on the same file. It is strongly recommended that, within a program, a unique channel be used to acquire all locks on any given file.

Some network filesystems permit file locking to be used with memory-mapped files only when the locked regions are page-aligned and a whole multiple of the underlying hardware's page size. Some network filesystems do not implement file locks on regions that extend past a certain position, often 2^{30} or 2^{31}. In general, great care should be taken when locking files that reside on network filesystems.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`FileLock(AsynchronousFileChannel channel, long position, long size, boolean shared)`	Initializes a new instance of this class.
protected	`FileLock(FileChannel channel, long position, long size, boolean shared)`	Initializes a new instance of this class.

Method Summary

Abstract Methods

Concrete Methods

Modifier and Type	Method	Description
`Channel`	`acquiredBy()`	Returns the channel upon whose file this lock was acquired.
`FileChannel`	`channel()`	Returns the file channel upon whose file this lock was acquired.
void	`close()`	This method invokes the `release()` method.
boolean	`isShared()`	Tells whether this lock is shared.
abstract boolean	`isValid()`	Tells whether or not this lock is valid.
boolean	`overlaps(long position, long size)`	Tells whether or not this lock overlaps the given lock range.

long	**position**()	Returns the position within the file of the first byte of the locked region.
abstract void	**release**()	Releases this lock.
long	**size**()	Returns the size of the locked region in bytes.
String	**toString**()	Returns a string describing the range, type, and validity of this lock.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

FileLock

```
protected FileLock(FileChannel channel,
                   long position,
                   long size,
                   boolean shared)
```

Initializes a new instance of this class.

Parameters:

channel - The file channel upon whose file this lock is held

position - The position within the file at which the locked region starts; must be non-negative

size - The size of the locked region; must be non-negative, and the sum position + size must be non-negative

shared - true if this lock is shared, false if it is exclusive

Throws:

IllegalArgumentException - If the preconditions on the parameters do not hold

FileLock

```
protected FileLock(AsynchronousFileChannel channel,
                   long position,
                   long size,
                   boolean shared)
```

Initializes a new instance of this class.

Parameters:

`channel - The channel upon whose file this lock is held`

`position - The position within the file at which the locked region starts; must be non-negative`

`size - The size of the locked region; must be non-negative, and the sum position + size must be non-negative`

`shared - true if this lock is shared, false if it is exclusive`

Throws:

`IllegalArgumentException - If the preconditions on the parameters do not hold`

Since:

`1.7`

Method Detail

channel

```
public final FileChannel channel()
```

Returns the file channel upon whose file this lock was acquired.

This method has been superseded by the `acquiredBy` method.

Returns:

`The file channel, or null if the file lock was not acquired by a file channel.`

acquiredBy

```
public Channel acquiredBy()
```

Returns the channel upon whose file this lock was acquired.

Returns:

The channel upon whose file this lock was acquired.

Since:

1.7

position

```
public final long position()
```

Returns the position within the file of the first byte of the locked region.

A locked region need not be contained within, or even overlap, the actual underlying file, so the value returned by this method may exceed the file's current size.

Returns:

The position

size

```
public final long size()
```

Returns the size of the locked region in bytes.

A locked region need not be contained within, or even overlap, the actual underlying file, so the value returned by this method may exceed the file's current size.

Returns:

The size of the locked region

isShared

```
public final boolean isShared()
```

Tells whether this lock is shared.

Returns:

true if lock is shared, false if it is exclusive

overlaps

```
public final boolean overlaps(long position,
                              long size)
```

Tells whether or not this lock overlaps the given lock range.

Parameters:

`position` - The starting position of the lock range

`size` - The size of the lock range

Returns:

`true if, and only if, this lock and the given lock range overlap by at least one byte`

isValid

```
public abstract boolean isValid()
```

Tells whether or not this lock is valid.

A lock object remains valid until it is released or the associated file channel is closed, whichever comes first.

Returns:

`true if, and only if, this lock is valid`

release

```
public abstract void release()
                     throws IOException
```

Releases this lock.

If this lock object is valid then invoking this method releases the lock and renders the object invalid. If this lock object is invalid then invoking this method has no effect.

Throws:

`ClosedChannelException` - If the channel that was used to acquire this lock is no longer open

`IOException` - If an I/O error occurs

close

```
public final void close()
                 throws IOException
```

This method invokes the `release()` method. It was added to the class so that it could be used in conjunction with the automatic resource management block construct.

Specified by:

close in interface AutoCloseable

Throws:

IOException

Since:

1.7

toString

```
public final String toString()
```

Returns a string describing the range, type, and validity of this lock.

Overrides:

toString in class Object

Returns:

A descriptive string

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.